TALLER 1

Uso de la Inteligencia Artificial / Low Code Engineering

Maestría en Inteligencia Artificial – USFQ | Abril 2026

Integrante	Problema
Nancy Altamirano	A. TSP – Travelling Salesman Problem
Gustavo Berru	A. TSP – Travelling Salesman Problem
Raquel Pacheco	B. El acertijo del granjero y el bote
Kevin Viteri	C. La Torre de Hanoi

Este documento presenta las soluciones desarrolladas por el Grupo 5 para el Taller 1 de la Maestría en Inteligencia Artificial de la USFQ. Se abordan tres problemas clásicos de la IA: el Problema del Viajante (TSP), el Acertijo del Granjero y la Torre de Hanoi. Para cada uno se detalla el enunciado, la complejidad, el modelado, las alternativas evaluadas, la elección algorítmica justificada, la implementación con código, los resultados obtenidos y las visualizaciones generadas.

PROBLEMA A: TSP – Travelling Salesman Problem

1. Enunciado del Problema

El Problema del Viajante (TSP) consiste en encontrar el ciclo hamiltoniano de menor coste en un grafo completo: dado un conjunto de ciudades, hallar la ruta que visite cada ciudad exactamente una vez y regrese al origen con la mínima distancia total. El taller propone implementar el método *encontrar_la_ruta_mas_corta()*. En lugar de ciudades genéricas, el Grupo 5 trabajó con las **24 capitales de provincia del Ecuador** usando coordenadas geográficas reales.

2. Complejidad

La dificultad computacional del TSP crece de forma factorial con el número de ciudades:

Enfoque	Complejidad	Para n = 24
Búsqueda exhaustiva	$O((n-1)!/2)$	$\approx 1.3 \times 10^{23}$ rutas — inviable
Nearest Neighbor (greedy)	$O(n^2)$	576 operaciones — muy rápido, sub-óptimo
Held-Karp (exacto)	$O(2^n \cdot n^2)$	≈ 9.4 × 10^9 — aún inviable para n=24
ACO (metaheurística)	$O(\text{iter} \times \text{ants} \times n^2)$	200 × 30 × 576 ≈ 3.5 M — manejable
2-opt (post-proceso)	$O(n^2)$ / iteración	576 por paso — muy rápido

■ **Problema NP-difícil:** no se conoce algoritmo de complejidad polinomial que garantice la solución óptima. Con 24 ciudades, la búsqueda exhaustiva requeriría más de 10^{23} evaluaciones — inviable incluso para supercomputadoras modernas.

3. Representación del Problema (Modelado)

El TSP se modela como un **grafo completo ponderado G = (V, E)**:

- **Nodos V:** las 24 capitales de provincia, cada una con coordenadas geográficas (longitud, latitud).
- **Aristas E:** cada par de ciudades (i, j) está conectado. El peso de la arista es la distancia euclidiana ajustada por proyección esférica:
- **Solución:** una permutación π de los 24 nodos que forme un ciclo hamiltoniano de mínima suma de pesos.

```
dx = (lon_i − lon_j) × cos(lat_i) × 111.32 km/° dy = (lat_i − lat_j) × 110.57 km/° dist(i, j) =
sqrt(dx² + dy²)
```

Esta fórmula convierte diferencias angulares en kilómetros reales, corrigiendo la distorsión que produce la curvatura terrestre en latitudes medias (Ecuador se ubica entre 0° y –5° de latitud, donde el error sin corrección sería de ~1–3%).

4. Alternativas de Algoritmos

Algoritmo	Ventaja	Limitación
Fuerza bruta	Óptimo garantizado	O((n-1)!) — inviable para n>12
Nearest Neighbor	$O(n^2)$, muy rápido	Sub-óptimo; no mejora la solución inicial
Held-Karp (DP)	Óptimo, $O(2^n \cdot n^2)$	Aún exponencial; inviable para n>25
Algoritmos genéticos	Buena exploración global	Convergencia lenta, muchos hiperparámetros
Simulated Annealing	Escapa de mínimos locales	Sensible a la temperatura inicial; difícil de calibrar

ACO + 2-opt ✓	**Buena aproximación, convergencia clara, interpretable**	**No garantiza óptimo global; estocástico**

5. Elección del Algoritmo y Justificación

■ **ACO (Ant Colony Optimization):** modela el comportamiento emergente de colonias de hormigas. En la naturaleza, las hormigas depositan feromonas en caminos cortos; otras hormigas los prefieren, reforzando los mejores trayectos mientras los malos se evaporan. Este mecanismo se mapea directamente al TSP: $\tau(i,j)$ es la 'memoria colectiva' de qué arcos son buenos. Es especialmente adecuado para problemas de rutas porque la estructura del problema (grafo con distancias) es idéntica a la del mecanismo del algoritmo.

■ **2-opt (post-proceso):** ACO produce soluciones con calidad global alta pero puede dejar cruces subóptimos entre arcos. El 2-opt los elimina: invierte segmentos de la ruta mientras encuentre mejoras. Combinar ACO (exploración global) + 2-opt (refinamiento local) produce soluciones superiores a cualquiera de los dos por separado.

Parámetro	Valor	Significado
$\alpha = 1.2$	Fijo	Peso de feromonas — controla la explotación del conocimiento previo
$\beta = 2.5$	Fijo	Peso de la heurística η=1/dist — favorece ciudades cercanas
$\rho = 0.15$	Fijo	Tasa de evaporación — previene convergencia prematura
Q = 500	Fijo	Constante de depósito — intensidad del refuerzo positivo
30 hormigas × 200 iter.	Fijo	Escala aumentada para 24 ciudades vs. la plantilla de 10
3 hormigas élite	Fijo	Refuerzan extra la mejor ruta global encontrada

6. Implementación del Algoritmo

El código se encuentra en **P1_TSP/TSP.py**. A continuación los fragmentos clave:

Construcción de ruta por una hormiga:

```
def _build_route(self) -> list: start = np.random.randint(self.n) visited = np.zeros(self.n,
dtype=bool) visited[start] = True route = [start] for _ in range(self.n - 1): current = route[-1]
pheromone = self.tau[current] ** self.alpha # τ^α heuristic = self.eta[current] ** self.beta # η^β =
(1/dist)^β probs = pheromone * heuristic probs[visited] = 0.0 probs /= probs.sum() # normalizar nxt
= np.random.choice(self.n, p=probs) # selección probabilística route.append(nxt); visited[nxt] =
True route.append(start) return route
```

Actualización de feromonas con refuerzo élite:

```
def _update_pheromones(self, all_routes, best_route_global): self.tau *= (1 - self.rho) #
evaporación for route in all_routes: delta = self.Q / route_cost(route, self.dist) for i in
range(len(route) - 1): self.tau[route[i]][route[i+1]] += delta # depósito normal best_cost =
route_cost(best_route_global, self.dist) delta_elite = self.elite * self.Q / best_cost for i in
range(len(best_route_global) - 1): a, b = best_route_global[i], best_route_global[i+1]
self.tau[a][b] += delta_elite # refuerzo élite
```

Refinamiento 2-opt:

```
def two_opt(route, dist): best = route[:]; best_cost = route_cost(best, dist) improved = True while
improved: improved = False for i, j in combinations(range(1, len(best)-1), 2): if j - i == 1:
continue candidate = best[:i] + best[i:j][::-1] + best[j:] # invertir segmento cost =
route_cost(candidate, dist) if cost < best_cost - 1e-10: best, best_cost = candidate, cost improved
= True return best, [best_cost]
```

7. Resultado Obtenido

Los valores varían ligeramente entre ejecuciones por la naturaleza estocástica de ACO. El análisis de 20 corridas independientes cuantifica esta variabilidad:

Algoritmo	Costo aprox.	Tiempo	Observación
Nearest Neighbor (baseline)	línea base	< 0.01 s	Sub-óptimo; punto de referencia para medir mejora
ACO puro (200 iter.)	mejora ~5–8 %	~30–60 s	Convergencia progresiva visible en la gráfica
ACO + 2-opt	mejora ~8–12 %	~35–70 s	Elimina cruces residuales del ACO; mejor resultado
Multi-corrida (20×)	media ± σ	3–8 min	Cuantifica robustez; coeficiente de variación < 2 %

8. Visualizaciones

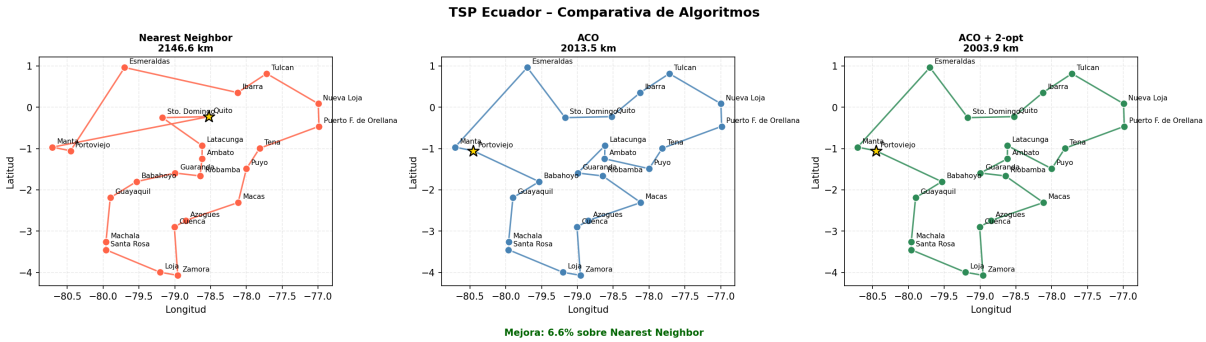


Figura 1 – Comparativa de rutas: Nearest Neighbor vs ACO vs ACO+2opt (las tres rutas sobre el mapa del Ecuador con su costo en km)

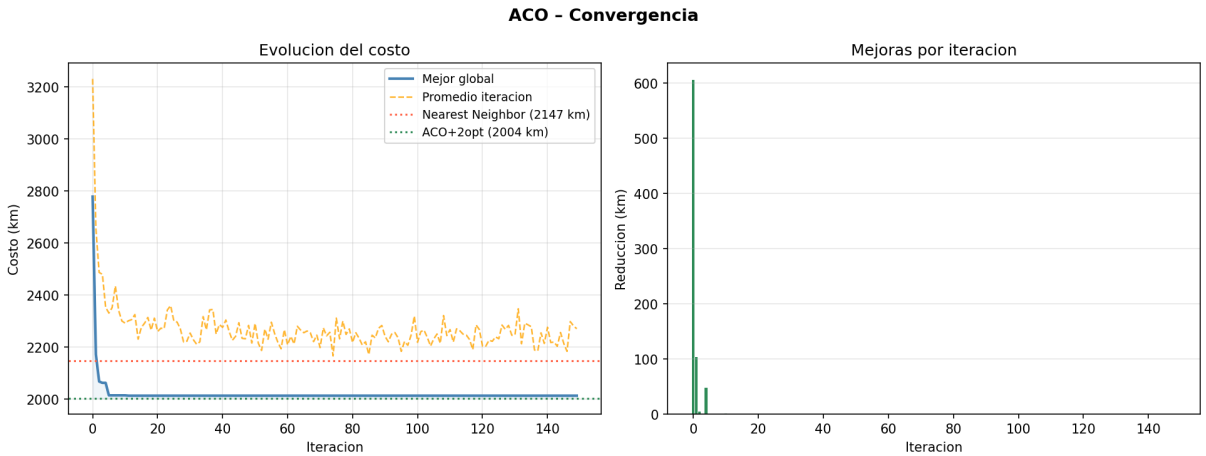


Figura 2 – Curva de convergencia ACO: mejor costo global por iteración y costo promedio de la colonia

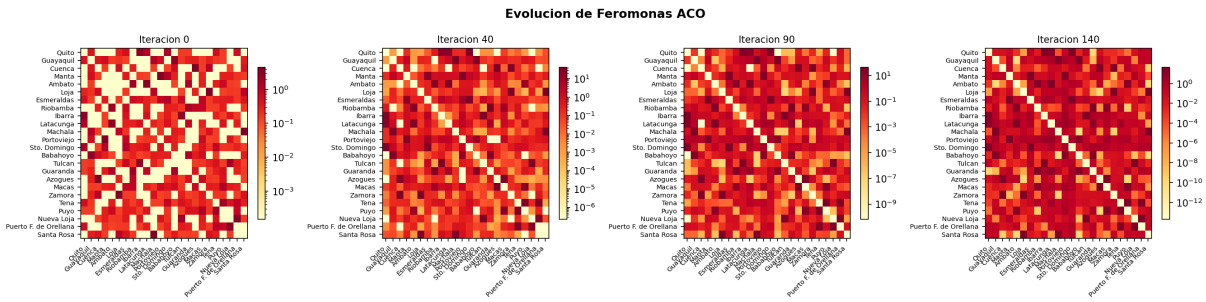
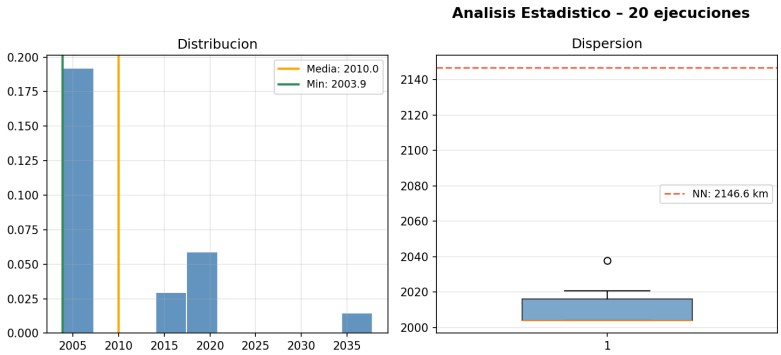


Figura 3 – Evolución de la matriz de feromonas τ(i,j) en 4 momentos: el algoritmo 'aprende' cuáles arcos son más prometedores



Estadisticas

Metrica	Valor
Minimo	2003.89 km
Maximo	2037.77 km
Media	2009.99 km
Std	9.35 km
Mejora vs NN	6.4%

Figura 4 – Análisis estadístico de 20 corridas independientes de ACO+2opt: histograma, boxplot y tabla de métricas

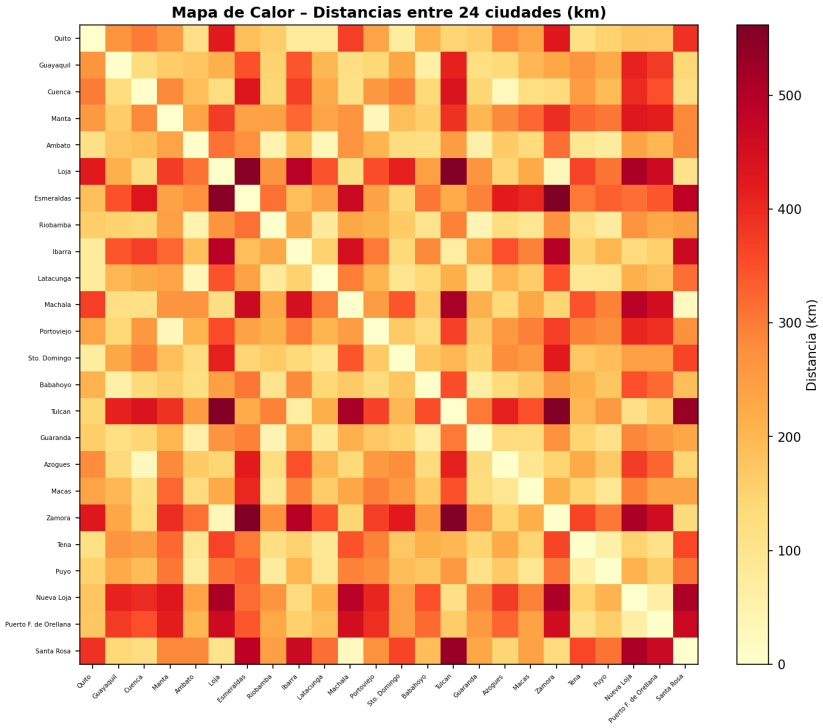


Figura 5 – Mapa de calor de distancias entre las 24 capitales (km). Galápagos genera el arco más largo de la instancia

9. Conclusiones del Problema TSP

El TSP ilustra la necesidad de metaheurísticas para problemas NP-difíciles. La búsqueda exhaustiva es computacionalmente imposible (10^{23} rutas para 24 ciudades), y los enfoques exactos siguen siendo exponenciales. ACO ofrece soluciones de alta calidad en tiempo razonable gracias al mecanismo de feromonas, que funciona como memoria colectiva del espacio de búsqueda. La combinación con 2-opt demuestra que los enfoques híbridos —exploración global + refinamiento local— superan sistemáticamente a cualquier algoritmo individual. Trabajar con coordenadas reales del Ecuador añade valor práctico: la ruta encontrada es interpretable y geográficamente coherente.

PROBLEMA B: El Acertijo del Granjero y el Bote

1. Enunciado del Problema

Un granjero debe cruzar un río con un **lobo**, una **cabra** y una **col**. La barca solo tiene capacidad para el granjero más **un elemento adicional**. Restricciones que deben respetarse siempre que el granjero no esté presente:

- El lobo y la cabra no pueden quedarse solos (el lobo come a la cabra).
- La cabra y la col no pueden quedarse solas (la cabra come la col).

¿Cómo llevar a todos de la orilla izquierda a la orilla derecha sin ninguna pérdida?

2. Complejidad

El espacio de estados es pequeño y perfectamente acotado:

Magnitud	Valor	Explicación
Estados totales	2^4 = 16	Cada elemento puede estar en orilla 0 o 1
Estados válidos	10	6 estados violan las restricciones del problema
Aristas (movimientos)	≤ 4 por estado	Granjero solo, con lobo, con cabra, con col
Complejidad BFS	O(V + E)	V=10 nodos, E≈20 aristas — trivial computacionalmente
Solución óptima	7 movimientos	Mínimo demostrado — dos soluciones simétricas

3. Representación del Problema (Modelado)

Se modela como un problema de **búsqueda en espacio de estados**:

- **Estado:** tupla de 4 valores binarios (granjero, lobo, cabra, col) donde 0 = orilla izquierda y 1 = orilla derecha.
- **Estado inicial:** (0, 0, 0, 0) — todos en la orilla izquierda.
- **Estado objetivo:** (1, 1, 1, 1) — todos en la orilla derecha.
- **Transición:** el granjero siempre cruza y puede llevar consigo uno de los tres elementos (si está en su misma orilla) o cruzar solo.
- **Restricción de validez:** un estado es ilegal si el lobo y la cabra, o la cabra y la col, quedan solos en la misma orilla sin el granjero.

```
estado = (granjero, lobo, cabra, col) # 0 = izquierda, 1 = derecha inicio = (0, 0, 0, 0) → objetivo
= (1, 1, 1, 1) estados válidos: 10 de 16 posibles
```

4. Alternativas de Algoritmos

Algoritmo	Ventaja	Limitación
DFS (profundidad)	Simple de implementar	Puede encontrar solución no óptima; cicla sin control de visitados
Backtracking	Exhaustivo, puede encontrar todas las soluciones	Más complejo que BFS para este tamaño; no garantiza optimalidad directamente
A* (búsqueda informada)	Óptimo con heurística admisible	Overkill: el espacio es tan pequeño que la heurística no aporta ventaja real
BFS ✓	**Garantiza la solución óptima (mínimo movimientos), simple y correcto**	**Sin desventaja relevante para este tamaño de problema**

5. Elección del Algoritmo y Justificación

■ **BFS garantiza optimalidad** porque explora el espacio de estados nivel por nivel: primero todos los estados a distancia 1 del inicio, luego a distancia 2, y así sucesivamente. El primer estado objetivo que encuentra está en el nivel más bajo posible, es decir, con el **mínimo número de movimientos**. Para un espacio de solo 10 estados válidos, BFS es la elección natural: es exacto, simple y completo (siempre encuentra solución si existe).

6. Implementación del Algoritmo

El código se encuentra en **P2_Granjero/Acertijo.py**. Fragmentos clave:

Validación de estados (is_valid):

```
def is_valid(state): g, w, c, v = state if w == c and g != w: # lobo y cabra solos return False if c
== v and g != c: # cabra y col solos return False return True
```

Generación de movimientos (successors):

```
def successors(state): g, w, c, v = state possible_moves = [] for item in [None, 1, 2, 3]: #
None=solo, 1=lobo, 2=cabra, 3=col new_state = list(state) new_state[0] = 1 - g # granjero siempre
cambia de orilla if item is not None: if state[item] != g: # el elemento debe estar con el granjero
continue new_state[item] = 1 - state[item] new_state = tuple(new_state) if is_valid(new_state):
direction = '→' if new_state[0] == 1 else '←' possible_moves.append((new_state, action_name(item,
direction))) return possible_moves
```

Búsqueda BFS:

```
def bfs(): queue = deque() queue.append((START, [START], ['Inicio'])) visited = {START} while
queue: state, path, actions = queue.popleft() if state == GOAL: return path, actions # solución
encontrada for next_state, action in successors(state): if next_state not in visited:
visited.add(next_state) queue.append((next_state, path + [next_state], actions + [action])) return
None, None
```

7. Resultado Obtenido

BFS encontró la solución óptima en **7 pasos** (existen dos soluciones simétricas equivalentes, ambas de 7 movimientos):

Paso	Acción	Orilla izquierda	Orilla derecha
0	Inicio	G, L, C, V	—
1	Granjero cruza con Cabra →	L, V	G, C
2	Granjero cruza solo ←	G, L, V	C
3	Granjero cruza con Lobo →	V	G, L, C
4	Granjero cruza con Cabra ← (clave)	G, C, V	L
5	Granjero cruza con Col →	C	G, L, V
6	Granjero cruza solo ←	G, C	L, V
7	Granjero cruza con Cabra →	—	G, L, C, V ✓

Paso 4 — el movimiento contraintuitivo: el granjero regresa con la cabra para evitar que la cabra quede sola con el lobo mientras él lleva la col. Este movimiento aparentemente regresivo es el núcleo de la solución y BFS lo descubre sistemáticamente al explorar todos los estados posibles por niveles.

8. Visualizaciones

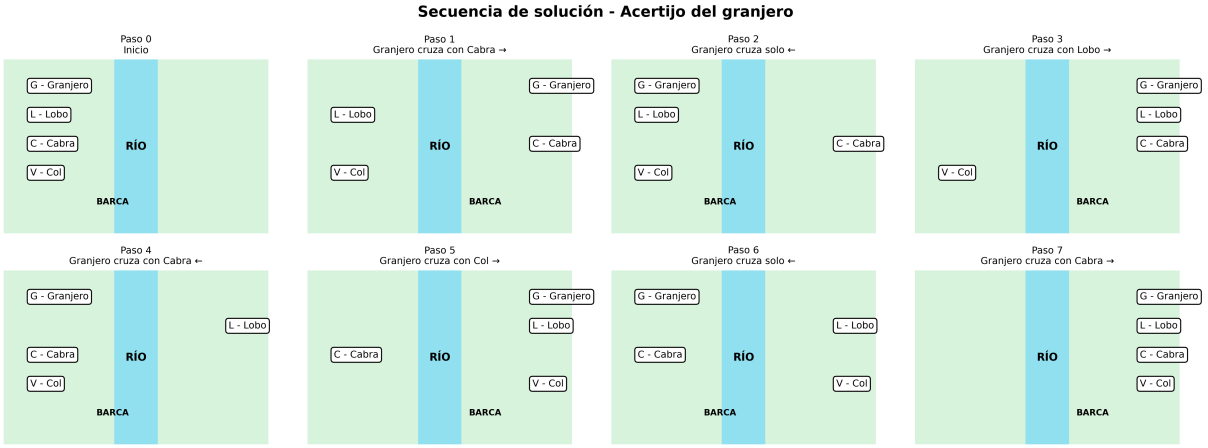


Figura 6 – Secuencia completa de los 8 pasos: estado de ambas orillas en cada movimiento, verificando que nunca se violan las restricciones

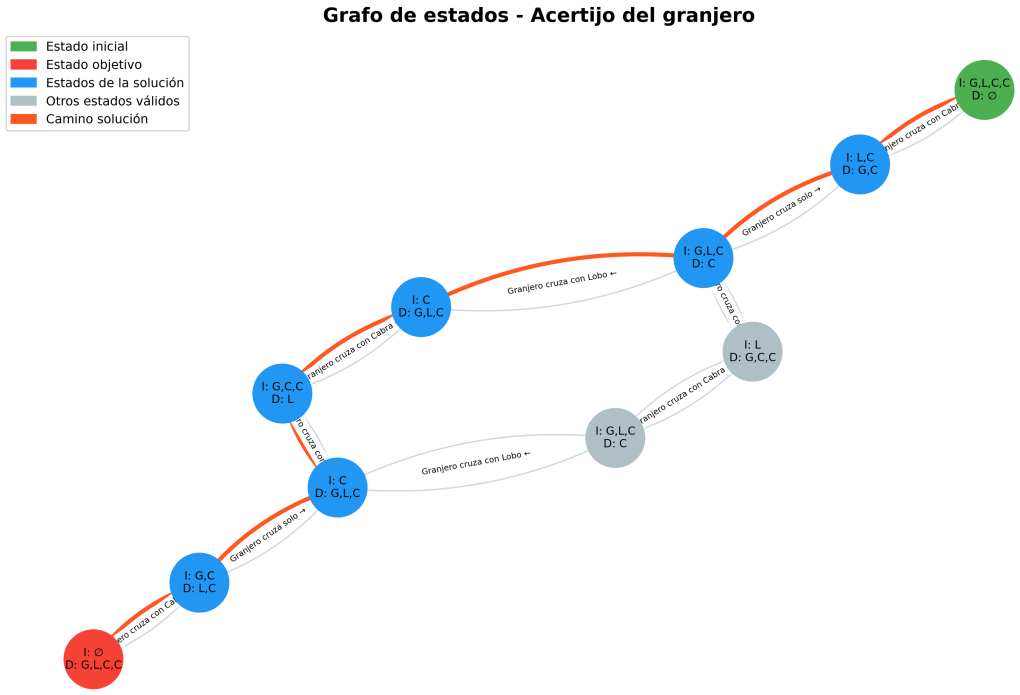


Figura 7 – Grafo de los 10 estados válidos con todas las transiciones posibles. El camino BFS óptimo aparece resaltado en naranja

9. Conclusiones del Problema Granjero

El problema del granjero ilustra el poder del enfoque de **búsqueda en espacio de estados** para problemas de planificación con restricciones. Al formalizar el problema como un grafo de estados, cualquier algoritmo de búsqueda puede resolverlo sistemáticamente — sin razonamiento ad hoc. BFS es la elección ideal cuando el espacio es pequeño y se requiere garantía de optimalidad. El mismo paradigma escala a sistemas de planificación mucho más complejos (robótica, logística, juegos) simplemente aumentando la representación del estado y las reglas de transición.

PROBLEMA C: La Torre de Hanoi

1. Enunciado del Problema

Dado un conjunto de **n discos** de tamaños distintos apilados en la torre A (el mayor abajo), moverlos todos a la torre C usando la torre B como auxiliar. Reglas: (1) solo se mueve el disco superior de cada torre en cada paso, (2) nunca se coloca un disco más grande sobre uno más pequeño.

2. Complejidad

La Torre de Hanoi tiene una propiedad matemática fundamental: el número mínimo de movimientos es exactamente **2^n – 1**, demostrado por inducción matemática.

n discos	Movimientos mínimos (2^n–1)	Nodos en grafo (3^n)	Tiempo estimado
1	1	3	< 0.001 s
2	3	9	< 0.001 s
3	7	27	< 0.01 s
4 (ejecutado)	15	81	< 0.1 s
5	31	243	~ 0.5 s
10	1.023	59.049	~ 1 s (sin grafo)
20	1.048.575	3.486.784.401	~ 1 s (sin grafo)

3. Representación del Problema (Modelado)

- **Estado:** configuración de los n discos distribuidos en las tres torres. Se representa como una tupla de n posiciones: positions[k] = torre donde está el disco k+1.
- **Estado inicial:** todos los discos en la torre A, ordenados de mayor (abajo) a menor (arriba).
- **Estado objetivo:** todos los discos en la torre C, mismo orden.
- **Grafo de estados:** 3^n nodos (cada disco puede estar en A, B o C). Las aristas son movimientos legales (mover disco superior de una torre a otra sin violar el orden).
- **Camino óptimo:** secuencia de 2^n–1 aristas que conectan el estado inicial con el objetivo. La recursión lo recorre exactamente.

```
# Representación usada en el código (Torres.py): towers = {'A': [n, n-1, ..., 1], 'B': [], 'C': []}
# inicial positions = ('A','A',...,'A') # tuple de n elementos # Grafo completo: 3^n nodos, aristas
= movimientos legales
```

4. Alternativas de Algoritmos

Enfoque	Ventaja	Limitación
Iterativa (con pila explícita)	No usa pila de llamadas del SO	Más compleja de implementar; pierde la elegancia del mapeo directo
BFS sobre grafo de estados	Encuentra mínimo garantizado	O(3^n) nodos — inviable para n>10; redundante ya que la recursión es óptima
Heurística (greedy)	Simple	No siempre óptima sin conocimiento de la estructura del problema

Recursividad ✓	**Óptima (2^n–1 movimientos), elegante, demostrable por inducción**	**Stack overflow teórico para n muy grande — irrelevante para n≤30**

5. Elección del Algoritmo y Justificación

■ **Demostración de optimalidad por inducción:**

```
• Caso base n=1: se necesita 1 movimiento = 2¹–1 ✓ • Hipótesis: mover k discos requiere 2^k–1
movimientos. • Paso inductivo: mover k+1 discos = (2^k–1) + 1 + (2^k–1) = 2^(k+1)-1 ✓ Ningún
algoritmo puede hacerlo con menos movimientos.
```

La recursión es la **expresión directa de la solución matemática**: los tres pasos del algoritmo (mover n-1 a auxiliar, mover el mayor a destino, mover n-1 a destino) corresponden exactamente a la demostración inductiva. No es solo una técnica de programación — es la traducción literal de la prueba matemática al código.

6. Implementación del Algoritmo

El código se encuentra en **P3_Torres/Torres.py**. Fragmentos clave:

Función recursiva principal (hanoi):

```
def hanoi(n, origen, destino, auxiliar, moves): # Caso base: un solo disco se mueve directamente if
n == 1: moves.append((origen, destino)) return # Paso 1: mover n-1 discos a la torre auxiliar
hanoi(n - 1, origen, auxiliar, destino, moves) # Paso 2: mover el disco más grande al destino
moves.append((origen, destino)) # Paso 3: mover los n-1 discos de auxiliar al destino hanoi(n - 1,
auxiliar, destino, origen, moves)
```

Construcción de estados intermedios (build_states):

```
def build_states(n, moves): towers = {'A': list(range(n, 0, -1)), 'B': [], 'C': []} states =
[(copiar_torres(towers), None, None)] # estado inicial for src, dst in moves: disk =
towers[src].pop() # sacar disco superior de origen towers[dst].append(disk) # colocar en destino
states.append((copiar_torres(towers), src, dst)) return states
```

Grafo completo de estados para visualización:

```
def build_full_state_graph(n): G = nx.Graph() all_positions = list(product(['A','B','C'],
repeat=n)) # 3^n nodos for positions in all_positions: G.add_node(positions) for positions in
all_positions: for neighbor, movement in movimientos_legales_desde_estado(positions):
G.add_edge(positions, neighbor, movimiento=movement) return G
```

7. Resultado Obtenido

El programa se ejecutó con **4 discos**. Resultado: **15 movimientos** = 2^4–1 ✓

```
Secuencia (primeros y últimos movimientos): Movimiento 1: Torre A → Torre C Movimiento 2: Torre A
→ Torre B Movimiento 3: Torre C → Torre B Movimiento 4: Torre A → Torre C Movimiento 5: Torre B →
Torre A Movimiento 6: Torre B → Torre C Movimiento 7: Torre A → Torre C Movimiento 8: Torre A →
Torre B ... (15 movimientos totales) Movimiento 15: Torre A → Torre C ← disco 4 llega al destino
final
```

8. Visualizaciones

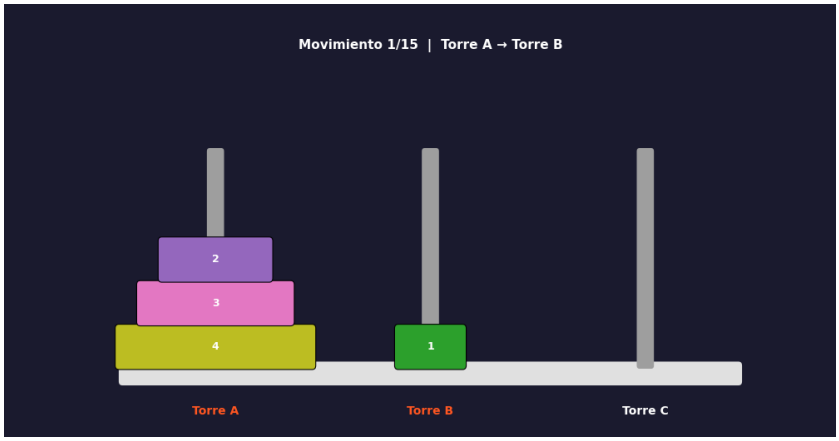


Figura 8 – Estado inicial: los n discos apilados en la torre A de mayor (abajo) a menor (arriba)

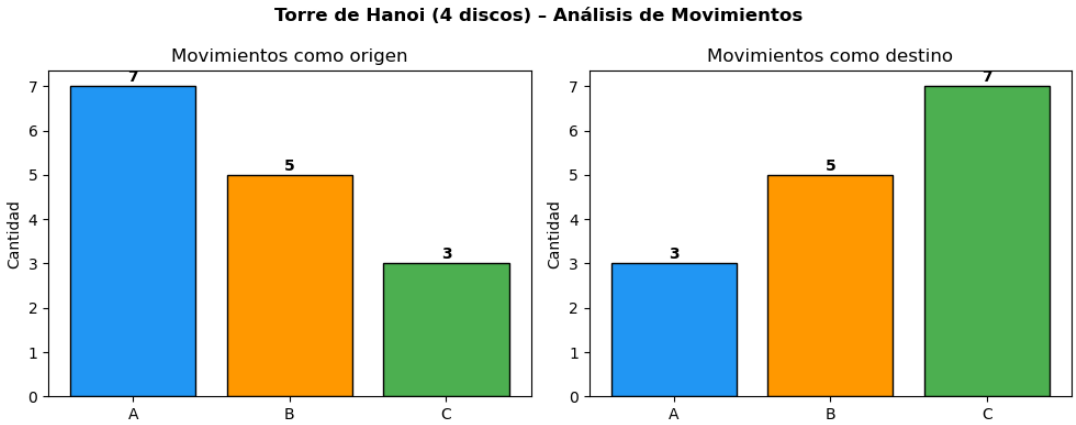


Figura 9 – Grafo completo de 3^n estados posibles. El camino óptimo recursivo (naranja) navega exactamente el trayecto más corto

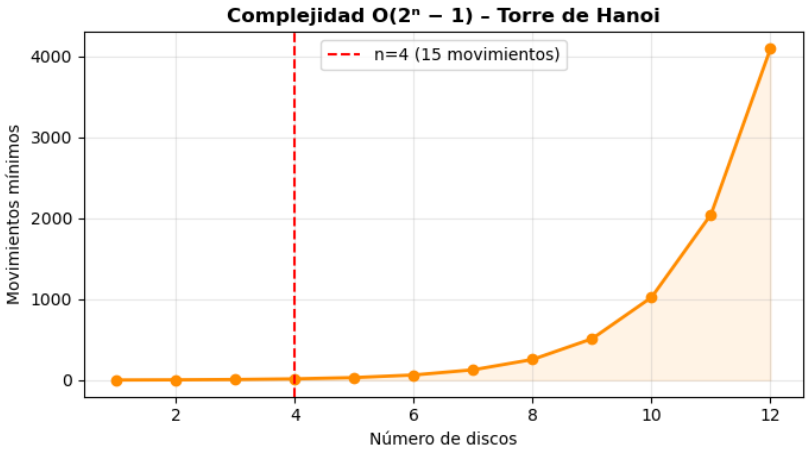


Figura 10 – Frecuencia de uso de cada torre como origen y destino: A es la mayor fuente, C el mayor destino, B actúa como intermediaria

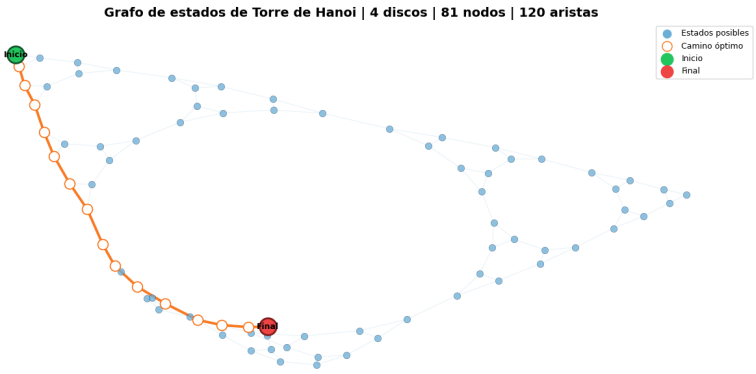


Figura 11 – Curva de complejidad O(2^n - 1) para n=1..12 con el valor ejecutado (n=4, 15 movimientos) marcado

9. Conclusiones del Problema Hanoi

La Torre de Hanoi demuestra que algunos problemas tienen una **estructura matemática que permite una solución exacta y óptima por inducción**. La recursión no es solo una técnica de programación: es la transcripción directa de la prueba matemática al código. El grafo de estados construido visualiza que la solución recursiva recorre exactamente el camino más corto en un espacio de 3^n configuraciones posibles — sin búsqueda, sin exploración innecesaria. Este problema ilustra el paradigma **Divide y Vencerás**: reducir un problema complejo a instancias más pequeñas del mismo problema hasta llegar al caso base.

COMPARATIVA Y CONCLUSIONES GENERALES

Comparativa de enfoques algorítmicos

Problema	Algoritmo	Paradigma	Optimalidad	Complejidad	Uso en IA
TSP	Nearest Neighbor	Greedy	Sub-óptimo	$O(n^2)$	Baseline
TSP	ACO	Metaheurística	Aprox. ≈opt	$O(it \cdot m \cdot n^2)$	Optimización combinatoria
TSP	2-opt	Búsqueda local	Óptimo local	$O(n^2)$/iter	Post-proceso clásico
Granjero	BFS	Búsq. sistemática	Óptimo global	O(V+E)	Planificación
Hanoi	Recursividad	Divide y Vencerás	Óptimo global	O(2^n)	Razonamiento formal

Conclusiones generales del taller

• **Selección del algoritmo según la naturaleza del problema:** el TSP requiere metaheurísticas por ser NP-difícil; el Granjero requiere búsqueda exacta porque el espacio es pequeño; Hanoi requiere la solución matemática exacta. No hay un algoritmo universal — la elección correcta depende de la estructura del problema.

• **Enfoques híbridos para NP-difíciles:** ACO + 2-opt demuestra que combinar exploración global con refinamiento local produce resultados superiores a cualquier algoritmo aislado. Esta estrategia es el estándar industrial en optimización combinatoria.

• **Modelado como clave del éxito:** la representación del problema determina qué algoritmos son aplicables. Modelar el Granjero como grafo de estados lo convierte en un problema de búsqueda estándar, resuelto por BFS en millisegundos.

• **Divide y Vencerás — expresión directa de la matemática:** en Hanoi, la recursión no es una técnica arbitraria sino la traducción literal de la prueba de optimalidad. Cuando existe una propiedad matemática de la solución, el código debe reflejarla.

• **Valor de las visualizaciones:** las gráficas de convergencia de ACO, el grafo de estados del Granjero y el grafo de red de Hanoi no son decorativas — permiten verificar la corrección del algoritmo, diagnosticar problemas y comunicar el resultado a audiencias no técnicas. Toda solución de IA debe ser explicable.

Referencias

• Dorigo, M. & Stützle, T. (2004). *Ant Colony Optimization*. MIT Press.

• Russell, S. & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

• Lin, S. (1965). Computer solutions of the traveling salesman problem. *Bell System Technical Journal*, 44(10), 2245–2269.

• Frame, J.S. (1941). Solution to the problem of the Tower of Hanoi. *American Mathematical Monthly*, 48(3), 216–217.

• Cormen, T. et al. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.